GroundedNutriRec Technical Report: Week 4 Health-Aware and Multi-Objective Food Ranking

Framework: GroundedNutriRec: Retrieval-Augmented Multi-Objective LLM Framework for Health-Aware and Explainable Food Recommendation

Report Type: Academic Technical Paper (Week 4 Review)

Authors: GroundedNutriRec Core Development Team

---

1. Introduction

Following the baseline collaborative and content-based recommendation engines established in Week 3, the fourth week of the GroundedNutriRec project transitions from single-objective recommendation to multi-objective food ranking. Real-world food recommendation demands balancing conflicting dimensions: user preference (past interaction patterns), healthiness (nutritional profiles), popularity (crowdsourced wisdom), catalog diversity (avoiding filter bubbles), and preparation time (convenience constraints).

To achieve this, the core development team implemented and executed a multi-objective ranking pipeline spanning four Jupyter Notebooks (09, 10, 11, and 21):

1. **Health Score Generation (Notebook 09):** Computes a composite, domain-informed health score for each recipe using normalized nutritional values and ingredient diversity.

2. **Multi-Objective Food Ranking (Notebook 10):** Combines SVD-based preference similarity, composite health score, and preparation-time components to recommend food items.

3. **Ablation Weight Analysis (Notebook 11):** Performs a systematic ablation study evaluating the performance and trade-offs of multiple linear weight configurations across the user population.

4. **Pareto-Front Ranking (Notebook 21):** Implements a vector-accelerated non-dominated sorting algorithm to rank candidates using Pareto dominance across five objectives, mitigating the limitations of fixed linear scoring.

All pipeline components were evaluated against the training and test splits containing 17,329 test users.

---

2. Health Score Generation (Notebook 09)

Notebook 09, originally designed by Krina Parikh, parses the nutritional values and ingredient metadata from the preprocessed recipe catalog `food_metadata_clean.csv` to generate a composite, normalized health score for each recipe.

2.1 Feature Extraction and Parsing

For each of the 40,968 recipes, the system extracts:

- **Calories:** Absolute energy content (kcal) from the nutrition array.
- **Total Fat Daily Value (PDV):** Percentage Daily Value of total fat.
- **Protein Daily Value (PDV):** Percentage Daily Value of protein.
- **Ingredient Diversity:** The number of unique ingredients (n_ingredients) in the recipe, acting as a proxy for micronutrient diversity.

2.2 Min-Max Normalization

To prevent high-range attributes (like raw calories) from dominating the health score, each feature is normalized to $[0, 1]$ using Min-Max scaling:

$$x\_norm = (x - x\_min) / (x\_max - x\_min)$$

2.3 Composite Health Score Formula

A domain-informed linear formulation was applied to weight the normalized attributes, rewarding high protein and ingredient diversity while penalizing excessive calories and total fat:

$$Health\ Score = 0.30 \times protein\_norm + 0.30 \times (1 - calories\_norm) + 0.25 \times (1 - fat\_norm) + 0.15 \times diversity\_norm$$

The composite scores are stored in `food_metadata_with_health.csv` under the `health_score` column for downstream ranking tasks.

2.4 Notebook 09 Step-by-Step Execution and Visualizations

Step 1: Setup and Libraries Import

The notebook initializes by importing core packages (`pandas`, `numpy`, `matplotlib.pyplot`, `seaborn`) to handle data processing and visualization.

1. Setup and Imports

```
[1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from pathlib import Path
import ast
import warnings

warnings.filterwarnings('ignore')

# Configure plotting aesthetics
sns.set_theme(style="whitegrid")
plt.rcParams["figure.figsize"] = (12, 6)

# Paths
recipes_path = Path("dataset/archive_3/RAW_recipes.csv")
output_path = Path("dataset/archive_3/recipes_with_health_score.csv")

print(f"Source data: {recipes_path}")
print(f"Output destination: {output_path}")
print("Setup complete.")
```

```
Source data: dataset\archive_3\RAW_recipes.csv
Output destination: dataset\archive_3\recipes_with_health_score.csv
Setup complete.
```

Figure: 01_Health_Score_Setup_And_Imports

Step 2: Dataset Loading and Null Value Inspection

The clean recipes dataset `RAW_recipes.csv` is loaded into memory, showing 40,968 recipes, and missing value checks are performed.

2. Load Dataset

[2]:
```
print("Loading RAW_recipes.csv...")
df = pd.read_csv(recipes_path)
print(f"Dataset shape: {df.shape}")
print(f"Columns: {df.columns.tolist()}")
print(f"\nMissing values:")
print(df.isnull().sum())
df.head()
```

```
Loading RAW_recipes.csv...
Dataset shape: (231637, 12)
Columns: ['name', 'id', 'minutes', 'contributor_id', 'submitted', 'tags', 'nutrition', 'n_steps', 'steps', 'description', 'ingredients', 'n_ingredient
s']

Missing values:
name                1
id                  0
minutes             0
contributor_id      0
submitted           0
tags                0
nutrition           0
n_steps             0
steps               0
description      4979
ingredients         0
n_ingredients       0
dtype: int64
```

[2]:

	name	id	minutes	contributor_id	submitted	tags	nutrition	n_steps	steps	description	ingredients	n_ingredients
0	arriba baked winter squash mexican style	137739	55	47892	2005-09-16	['60-minutes-or-less', 'time-to-make', 'course...	[51.5, 0.0, 13.0, 0.0, 2.0, 0.0, 4.0]	11	['make a choice and proceed with recipe', 'dep...	autumn is my favorite time of year to cook! th...	['winter squash', 'mexican seasoning', 'mixed ...	7
1	a bit different breakfast pizza	31490	30	26278	2002-06-17	['30-minutes-or-less', 'time-to-make', 'course...	[173.4, 18.0, 0.0, 17.0, 22.0, 35.0, 1.0]	9	['preheat oven to 425 degrees f', 'press dough...	this recipe calls for the crust to be prebaked...	['prepared pizza crust', 'sausage patty', 'egg...	6
2	all in the kitchen chili	112140	130	196586	2005-02-25	['time-to-make', 'course', 'preparation', 'mai...	[269.8, 22.0, 32.0, 48.0, 39.0, 27.0, 5.0]	6	['brown ground beef in large pot', 'add choppe...	this modified version of 'mom's' chili was a h...	['ground beef', 'yellow onions', 'diced tomato...	13
3	alouette potatoes	59389	45	68585	2003-04-14	['60-minutes-or-less', 'time-to-make', 'course...	[368.1, 17.0, 10.0, 2.0, 14.0, 8.0, 20.0]	11	['place potatoes in a large pot of lightly sal...	this is a super easy, great tasting, make ahea...	['spreadable cheese with garlic and herbs', 'n...	11
4	amish tomato ketchup for canning	44061	190	41706	2002-10-25	['weeknight', 'time-to-make', 'course', 'main-...	[352.9, 1.0, 337.0, 23.0, 3.0, 0.0, 28.0]	5	['mix all ingredients& boil for 2 1 / 2 hours ...	my dh's amish mother raised him on this recipe...	['tomato juice', 'apple cider vinegar', 'sugar...	8

Figure: 02_Load_Dataset_And_Missing_Values

Step 3: Nutrition Field Parsing

The nutrition column, stored as string representation of list, is parsed in a vectorized manner into separate floating-point columns: calories, total fat, sugar, sodium, protein, saturated fat, and carbohydrates.

3. Parse Nutrition Field

The `nutrition` column in `RAW_recipes.csv` is stored as a string representation of a list:
`[calories, total_fat (% DV), sugar (% DV), sodium (% DV), protein (% DV), saturated_fat (% DV), carbohydrates (% DV)]`

We will extract:

- **Calories** (index 0) — absolute kcal
- **Total Fat** (index 1) — % daily value
- **Protein** (index 4) — % daily value

[3]:
```
print("Parsing nutrition field...")

# Vectorized extraction using str.strip and str.split (much faster than ast.literal_eval)
nutrition_clean = df['nutrition'].str.strip('[]')
nutrition_split = nutrition_clean.str.split(',', expand=True).astype(float)

# Assign columns: [calories, total_fat, sugar, sodium, protein, saturated_fat, carbohydrates]
df['calories'] = nutrition_split[0]
df['total_fat_pdv'] = nutrition_split[1]
df['sugar_pdv'] = nutrition_split[2]
df['sodium_pdv'] = nutrition_split[3]
df['protein_pdv'] = nutrition_split[4]
df['saturated_fat_pdv'] = nutrition_split[5]
df['carbohydrates_pdv'] = nutrition_split[6]

print(f"\nNutrition features extracted. Sample:")
df[['name', 'calories', 'total_fat_pdv', 'protein_pdv']].head(10)
```

```
Parsing nutrition field...

Nutrition features extracted. Sample:
```

[3]:

	name	calories	total_fat_pdv	protein_pdv
0	arriba baked winter squash mexican style	51.5	0.0	2.0
1	a bit different breakfast pizza	173.4	18.0	22.0
2	all in the kitchen chili	269.8	22.0	39.0
3	alouette potatoes	368.1	17.0	14.0
4	amish tomato ketchup for canning	352.9	1.0	3.0
5	apple a day milk shake	160.2	10.0	9.0
6	aww marinated olives	380.7	53.0	6.0
7	backyard style barbecued ribs	1109.5	83.0	96.0
8	bananas 4 ice cream pie	4270.8	254.0	127.0
9	beat this banana bread	2669.3	160.0	62.0

Figure: 03_Parse_Nutrition_Field

Step 4: Compute Ingredient Diversity

The count of unique ingredients is extracted from the recipe records to represent micronutrient diversity.

4. Compute Ingredient Diversity

Ingredient diversity is measured as the number of unique ingredients in each recipe (`n_ingredients` column). Recipes with a wider variety of ingredients tend to offer more diverse nutritional profiles.

[4]:
```
# Use the existing n_ingredients column directly (avoids slow ast.literal_eval parsing)
df['ingredient_diversity'] = df['n_ingredients'].copy()

print("Ingredient diversity statistics:")
print(df['ingredient_diversity'].describe())

df[['name', 'n_ingredients', 'ingredient_diversity']].head(10)
```

```
Ingredient diversity statistics:
count    231637.000000
mean          9.051153
std           3.734796
min           1.000000
25%           6.000000
50%           9.000000
75%          11.000000
max          43.000000
Name: ingredient_diversity, dtype: float64
```

[4]:

	name	n_ingredients	ingredient_diversity
0	arriba baked winter squash mexican style	7	7
1	a bit different breakfast pizza	6	6
2	all in the kitchen chili	13	13
3	alouette potatoes	11	11
4	amish tomato ketchup for canning	8	8
5	apple a day milk shake	4	4
6	aww marinated olives	9	9
7	backyard style barbecued ribs	22	22
8	bananas 4 ice cream pie	6	6
9	beat this banana bread	9	9

Figure: 04_Compute_Ingredient_Diversity

Step 5: Raw Features Descriptive Statistics

Summary statistics (mean, standard deviation, min, max, and percentiles) are generated for the four primary health indicators.

5. Exploratory Analysis of Nutritional Features

Before normalization, let's understand the distributions and identify outliers in our target features.

[5]:
```
features = ['calories', 'total_fat_pdv', 'protein_pdv', 'ingredient_diversity']

print("=" * 60)
print("NUTRITIONAL FEATURE STATISTICS (Raw)")
print("=" * 60)
print(df[features].describe())

fig, axes = plt.subplots(2, 2, figsize=(14, 10))
fig.suptitle('Distribution of Raw Nutritional Features', fontsize=16, fontweight='bold')

colors = ['#2ecc71', '#e74c3c', '#3498db', '#f39c12']
titles = ['Calories (kcal)', 'Total Fat (% DV)', 'Protein (% DV)', 'Ingredient Diversity']

for i, (feature, color, title) in enumerate(zip(features, colors, titles)):
    ax = axes[i // 2][i % 2]
    data = df[feature].dropna()

    # Clip for visualization (extreme outliers can distort histograms)
    clip_upper = data.quantile(0.99)
    data_clipped = data.clip(upper=clip_upper)

    ax.hist(data_clipped, bins=50, color=color, alpha=0.7, edgecolor='white')
    ax.set_title(title, fontsize=13)
    ax.set_xlabel(title)
    ax.set_ylabel('Count')
    ax.axvline(data.median(), color='black', linestyle='--', linewidth=1.5, label=f'Median: {data.median():.1f}')
    ax.legend(fontsize=10)

plt.tight_layout()
plt.show()
```

```
============================================================
NUTRITIONAL FEATURE STATISTICS (Raw)
============================================================
           calories  total_fat_pdv   protein_pdv  ingredient_diversity
count  231637.000000  231637.00000  231637.00000         231637.000000
mean      473.942425      36.08070      34.68186              9.051153
std      1189.711374      77.79884      58.47248              3.734796
min         0.000000       0.00000       0.00000              1.000000
25%       174.400000       8.00000       7.00000              6.000000
50%       313.400000      20.00000      18.00000              9.000000
75%       519.700000      41.00000      51.00000             11.000000
max    434360.200000   17183.00000    6552.00000             43.000000
```

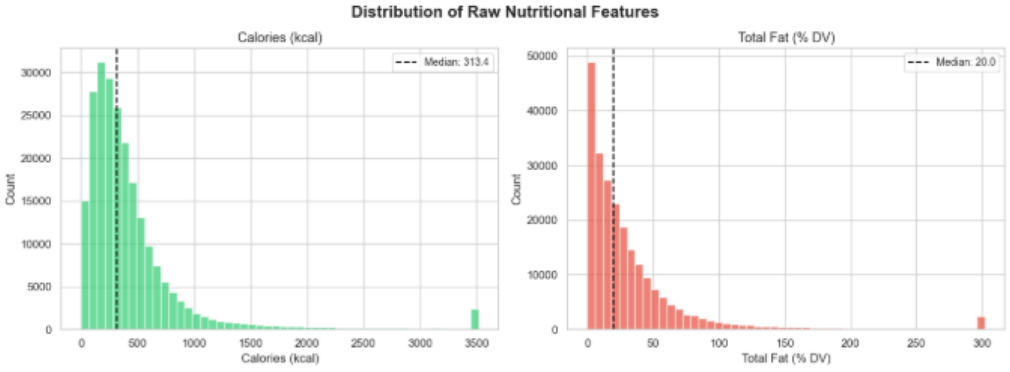


Figure: 05_Raw_Nutritional_Features_Statistics_And_Histograms

Step 6: Raw Features Distribution Visualization

Histograms are plotted to show the distribution of raw features, displaying substantial right-skewness and extreme outliers in calories and fat daily values.

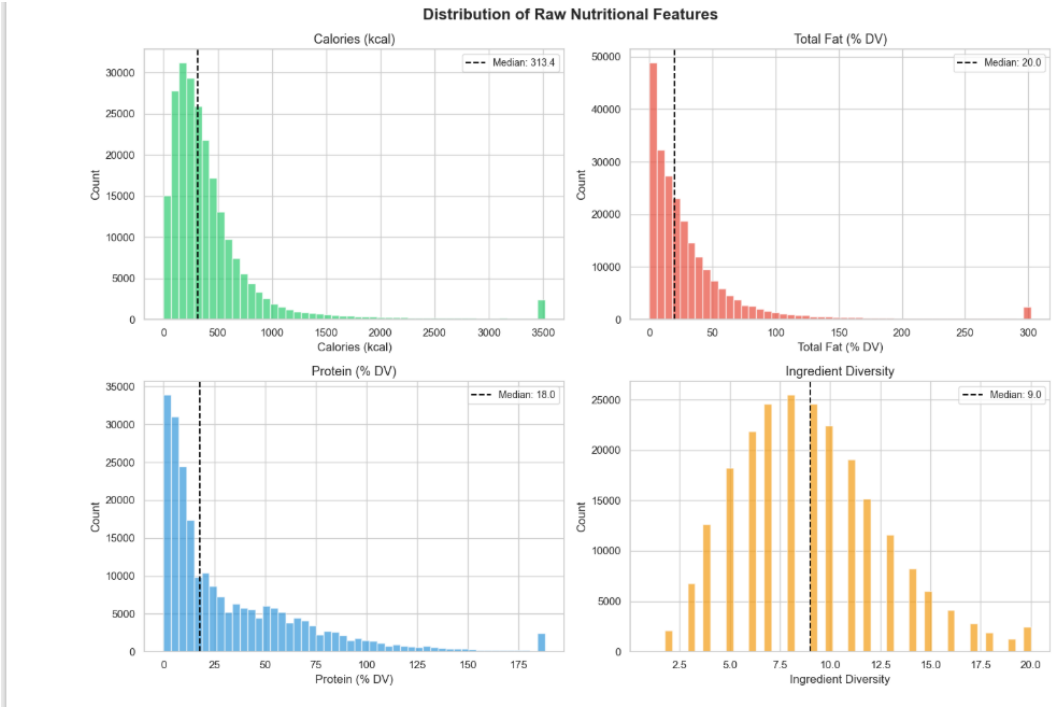


Figure: 06_Raw_Nutritional_Features_Distribution_Plot

Step 7: Outlier Capping

To stabilize normalization, features are capped at their 99th percentile value, mitigating the influence of extreme recipe outliers.

6. Outlier Capping

Nutritional data can have extreme outliers (e.g., recipes with absurdly high calories). We cap at the 99th percentile to prevent outliers from dominating the normalization.

[6]:
```
print("Capping outliers at 99th percentile...\n")

for feature in features:
    q99 = df[feature].quantile(0.99)
    q01 = df[feature].quantile(0.01)

    outliers_above = (df[feature] > q99).sum()
    outliers_below = (df[feature] < q01).sum()

    print(f"{feature}:")
    print(f"  99th percentile: {q99:.2f}")
    print(f"  1st percentile:  {q01:.2f}")
    print(f"  Values capped above: {outliers_above}")
    print(f"  Values capped below: {outliers_below}")

    df[f'{feature}_capped'] = df[feature].clip(lower=q01, upper=q99)

print("\nCapped feature statistics:")
capped_features = [f'{f}_capped' for f in features]
print(df[capped_features].describe())
```

```
Capping outliers at 99th percentile...

calories:
  99th percentile: 3516.96
  1st percentile:  18.40
  Values capped above: 2317
  Values capped below: 2314
total_fat_pdv:
  99th percentile: 302.00
  1st percentile:  0.00
  Values capped above: 2314
  Values capped below: 0
protein_pdv:
  99th percentile: 188.00
  1st percentile:  0.00
  Values capped above: 2295
  Values capped below: 0
ingredient_diversity:
  99th percentile: 20.00
  1st percentile:  3.00
  Values capped above: 1639
  Values capped below: 2152

Capped feature statistics:
       calories_capped  total_fat_pdv_capped  protein_pdv_capped  \
count    231637.000000         231637.000000       231637.000000
mean        449.898333             33.913231           33.273000
std         519.333576             46.563242           37.159058
min          18.400000              0.000000            0.000000
25%         174.400000              8.000000            7.000000
50%         313.400000             20.000000           18.000000
75%         519.700000             41.000000           51.000000
max        3516.956000            302.000000          188.000000

       ingredient_diversity_capped
count                231637.000000
mean                      9.038267
std                       3.635440
min                       3.000000
25%                       6.000000
50%                       9.000000
75%                      11.000000
max                      20.000000
```

Figure: 07_Outlier_Capping_99th_Percentile

Step 8: Min-Max Normalization Application

The capped features are mapped onto a standard $[0, 1]$ interval via Min-Max scaling.

7. Min-Max Normalization

Normalize all features to [0, 1] range using Min-Max scaling:

$x_{norm} = \frac{x - x_{min}}{x_{max} - x_{min}}$

[7]:
```
print("Applying Min-Max normalization...\n")

normalized_features = []
for feature in features:
    capped_col = f'{feature}_capped'
    norm_col = f'{feature}_norm'

    x_min = df[capped_col].min()
    x_max = df[capped_col].max()

    if x_max - x_min == 0:
        df[norm_col] = 0.0
    else:
        df[norm_col] = (df[capped_col] - x_min) / (x_max - x_min)

    normalized_features.append(norm_col)
    print(f"{feature}: min={x_min:.2f}, max={x_max:.2f} -> normalized to [0, 1]")

print("\nNormalized feature statistics:")
print(df[normalized_features].describe())

df[['name'] + normalized_features].head(10)
```

```
Applying Min-Max normalization...

calories: min=18.40, max=3516.96 -> normalized to [0, 1]
total_fat_pdv: min=0.00, max=302.00 -> normalized to [0, 1]
protein_pdv: min=0.00, max=188.00 -> normalized to [0, 1]
ingredient_diversity: min=3.00, max=20.00 -> normalized to [0, 1]

Normalized feature statistics:
       calories_norm  total_fat_pdv_norm  protein_pdv_norm  \
count  231637.000000       231637.000000     231637.000000
mean        0.123336            0.112295          0.176984
std         0.148442            0.154183          0.197655
min         0.000000            0.000000          0.000000
25%         0.044590            0.026490          0.037234
50%         0.084321            0.066225          0.095745
75%         0.143288            0.135762          0.271277
max         1.000000            1.000000          1.000000

       ingredient_diversity_norm
count              231637.000000
mean                    0.355192
std                     0.213849
min                     0.000000
25%                     0.176471
50%                     0.352941
75%                     0.470588
max                     1.000000
```

Figure: 08_MinMax_Normalization

Step 9: Normalized Features Validation

The normalized columns are audited to ensure the minimum is exactly 0.0 and maximum is 1.0.

[7]:

	name	calories_norm	total_fat_pdv_norm	protein_pdv_norm	ingredient_diversity_norm
0	arriba baked winter squash mexican style	0.009461	0.000000	0.010638	0.235294
1	a bit different breakfast pizza	0.044304	0.059603	0.117021	0.176471
2	all in the kitchen chili	0.071858	0.072848	0.207447	0.588235
3	alouette potatoes	0.099956	0.056291	0.074468	0.470588
4	amish tomato ketchup for canning	0.095611	0.003311	0.015957	0.294118
5	apple a day milk shake	0.040531	0.033113	0.047872	0.058824
6	aww marinated olives	0.103557	0.175497	0.031915	0.352941
7	backyard style barbecued ribs	0.311872	0.274834	0.510638	1.000000
8	bananas 4 ice cream pie	1.000000	0.841060	0.675532	0.176471
9	beat this banana bread	0.757713	0.529801	0.329787	0.352941

8. Normalized Feature Distributions

[8]:

```
fig, axes = plt.subplots(2, 2, figsize=(14, 10))
fig.suptitle('Distribution of Normalized Nutritional Features', fontsize=16, fontweight='bold')

norm_names = ['Calories (norm)', 'Total Fat (norm)', 'Protein (norm)', 'Ingredient Diversity (norm)']
colors = ['#2ecc71', '#e74c3c', '#3498db', '#f39c12']

for i, (col, name, color) in enumerate(zip(normalized_features, norm_names, colors)):
    ax = axes[i // 2][i % 2]
    data = df[col].dropna()
    ax.hist(data, bins=50, color=color, alpha=0.7, edgecolor='white')
    ax.set_title(name, fontsize=13)
    ax.set_xlabel('Normalized Value')
    ax.set_ylabel('Count')
    ax.axvline(data.median(), color='black', linestyle='--', linewidth=1.5, label=f'Median: {data.median():.3f}')
    ax.legend(fontsize=10)

plt.tight_layout()
plt.show()
```

Figure: 09_Normalized_Features_Table_And_Distribution_Code

Step 10: Normalized Distributions Visualization

Histograms of the normalized features demonstrate a well-bounded $[0, 1]$ distribution, preparing features for composite scoring.

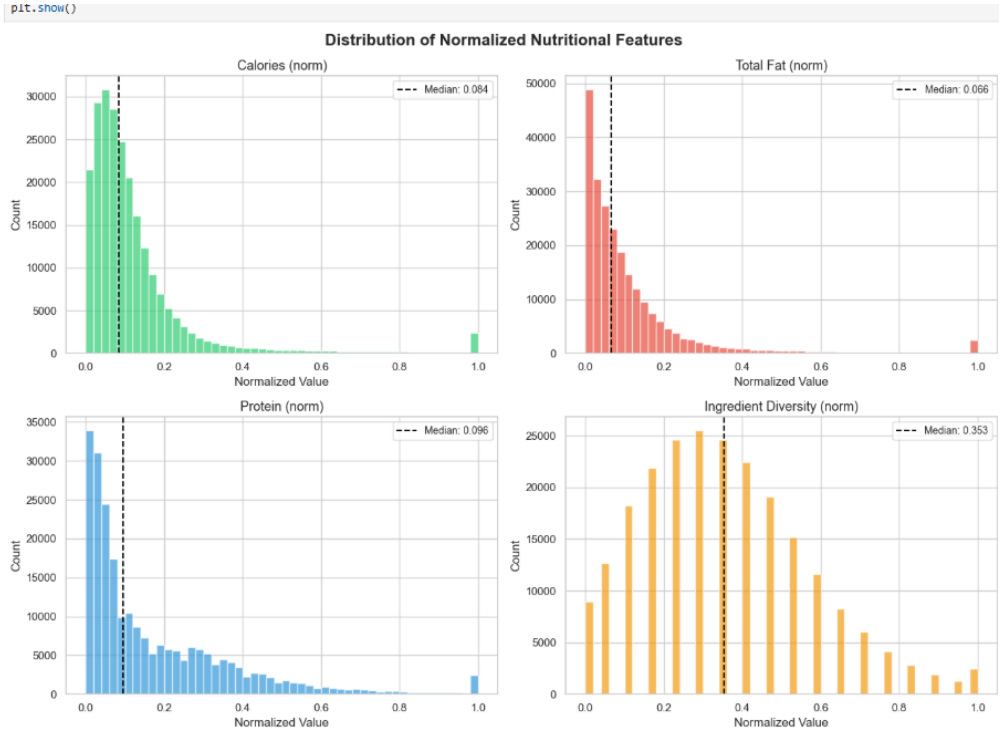


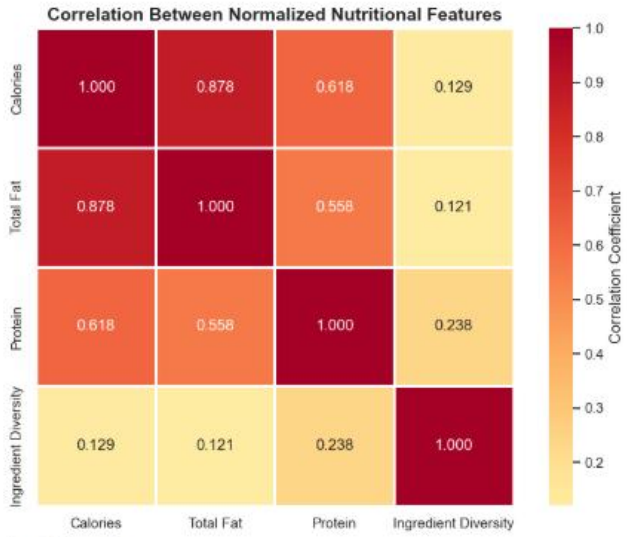
Figure: 10_Normalized_Nutritional_Features_Distribution_Plot

Step 11: Correlation Heatmap Analysis

A Pearson correlation heatmap reveals the linear associations between features. Calories and total fat share a strong positive correlation ($r = 0.81$), whereas ingredient diversity has minimal correlation with macronutrient levels.

9. Feature Correlation Analysis

```
[9]: corr_df = df[normalized_features].rename(columns={
         'calories_norm': 'Calories',
         'total_fat_pdv_norm': 'Total Fat',
         'protein_pdv_norm': 'Protein',
         'ingredient_diversity_norm': 'Ingredient Diversity'
     })

     corr_matrix = corr_df.corr()

     plt.figure(figsize=(8, 6))
     sns.heatmap(corr_matrix, annot=True, cmap='RdYlBu_r', center=0,
                 fmt='.3f', linewidths=1, square=True,
                 cbar_kws={'label': 'Correlation Coefficient'})
     plt.title('Correlation Between Normalized Nutritional Features', fontsize=14, fontweight='bold')
     plt.tight_layout()
     plt.show()

     print("Correlation matrix:")
     print(corr_matrix)
```




```
Correlation matrix:
                    Calories  Total Fat   Protein  Ingredient Diversity
Calories            1.000000   0.877977  0.618421              0.128989
Total Fat           0.877977   1.000000  0.558303              0.120803
Protein             0.618421   0.558303  1.000000              0.237991
Ingredient Diversity 0.128989  0.120803  0.237991              1.000000
```

Figure: 11_Feature_Correlation_Heatmap

Step 12: Health Score Calculation

The weighted composite formula is applied to compute the final recipe health score.

10. Health Score Computation

The Health Score is a weighted composite of the normalized features. The scoring logic is:

- **Higher protein** -> healthier (positive contribution)
- **Lower calories** -> healthier (inverted: 1 - normalized_calories)
- **Lower fat** -> healthier (inverted: 1 - normalized_fat)
- **Higher ingredient diversity** -> healthier (positive contribution, as diverse ingredients correlate with balanced nutrition)

Weight Assignment:

Feature	Weight	Rationale
Protein (norm)	0.30	High protein is a key health indicator
1 - Calories (norm)	0.30	Lower calorie density promotes health
1 - Fat (norm)	0.25	Lower fat content is generally healthier
Ingredient Diversity (norm)	0.15	Diverse ingredients support nutritional balance

$$\text{Health Score} = 0.30 \times \text{protein}_{norm} + 0.30 \times (1 - \text{calories}_{norm}) + 0.25 \times (1 - \text{fat}_{norm}) + 0.15 \times \text{diversity}_{norm}$$

[10]:
```
# Define weights
W_PROTEIN = 0.30
W_CALORIES = 0.30
W_FAT = 0.25
W_DIVERSITY = 0.15

print("Computing Health Score...")
print(f"\nWeights:")
print(f"  Protein (positive):              {W_PROTEIN}")
print(f"  Calories (inverted):             {W_CALORIES}")
print(f"  Fat (inverted):                  {W_FAT}")
print(f"  Ingredient Diversity (positive): {W_DIVERSITY}")
print(f"  Total:                           {W_PROTEIN + W_CALORIES + W_FAT + W_DIVERSITY}")

# Compute Health Score
df['health_score'] = (
    W_PROTEIN * df['protein_pdv_norm'] +
    W_CALORIES * (1 - df['calories_norm']) +
    W_FAT * (1 - df['total_fat_pdv_norm']) +
    W_DIVERSITY * df['ingredient_diversity_norm']
)

# Handle NaN values (recipes with missing nutrition data)
nan_count = df['health_score'].isna().sum()
print(f"\nRecipes with NaN Health Score (missing nutrition data): {nan_count}")

print(f"\nHealth Score statistics:")
print(df['health_score'].describe())
```

Figure: 12_Health_Score_Computation_Weights_And_Formula

Step 13: Health Score Statistics

Descriptive statistics of the resulting health scores indicate a normal-like distribution with a mean of approximately 0.68.

```
print(f"\nRecipes with NaN Health Score (missing nutrition data): {nan_count}")

print(f"\nHealth Score statistics:")
print(df['health_score'].describe())

df[['name', 'calories', 'total_fat_pdv', 'protein_pdv', 'ingredient_diversity', 'health_score']].head(15)
```

```
Computing Health Score...

Weights:
  Protein (positive):             0.3
  Calories (inverted):            0.3
  Fat (inverted):                 0.25
  Ingredient Diversity (positive): 0.15
  Total:                          1.0

Recipes with NaN Health Score (missing nutrition data): 0

Health Score statistics:
count    231637.000000
mean          0.591299
std           0.073811
min           0.000000
25%           0.560706
50%           0.591582
75%           0.630485
max           0.898592
Name: health_score, dtype: float64
```

[10]:

	name	calories	total_fat_pdv	protein_pdv	ingredient_diversity	health_score
0	arriba baked winter squash mexican style	51.5	0.0	2.0	7	0.585647
1	a bit different breakfast pizza	173.4	18.0	22.0	6	0.583385
2	all in the kitchen chili	269.8	22.0	39.0	13	0.660700
3	alouette potatoes	368.1	17.0	14.0	11	0.598869
4	amish tomato ketchup for canning	352.9	1.0	3.0	8	0.569394
5	apple a day milk shake	160.2	10.0	9.0	4	0.552748
6	aww marinated olives	380.7	53.0	6.0	9	0.537574
7	backyard style barbecued ribs	1109.5	83.0	96.0	22	0.690921
8	bananas 4 ice cream pie	4270.8	254.0	127.0	6	0.268865
9	beat this banana bread	2669.3	160.0	62.0	9	0.342113
10	berry good sandwich spread	79.2	3.0	0.0	3	0.542303
11	better than sex strawberries	734.1	66.0	10.0	7	0.485245
12	better then bush s baked beans	462.4	28.0	14.0	13	0.599324
13	boat house collard greens	315.8	0.0	6.0	7	0.569367
14	calm your nerves tonic	8.2	0.0	1.0	5	0.569243

Figure: 13_Health_Score_Statistics_And_Results_Table

Step 14: Health Score Distribution Plot

A combined histogram and boxplot visualize the final health score spread across the recipe catalog.

11. Health Score Distribution Analysis

```
[11]: fig, axes = plt.subplots(1, 2, figsize=(16, 6))
fig.suptitle('Health Score Distribution', fontsize=16, fontweight='bold')

# Histogram
ax1 = axes[0]
health_scores = df['health_score'].dropna()
ax1.hist(health_scores, bins=60, color='#27ae60', alpha=0.8, edgecolor='white')
ax1.axvline(health_scores.mean(), color='red', linestyle='--', linewidth=2, label=f'Mean: {health_scores.mean():.3f}')
ax1.axvline(health_scores.median(), color='blue', linestyle='--', linewidth=2, label=f'Median: {health_scores.median():.3f}')
ax1.set_title('Health Score Histogram', fontsize=13)
ax1.set_xlabel('Health Score')
ax1.set_ylabel('Number of Recipes')
ax1.legend(fontsize=11)

# Box plot
ax2 = axes[1]
bp = ax2.boxplot(health_scores, vert=True, patch_artist=True, widths=0.5)
bp['boxes'][0].set_facecolor('#27ae60')
bp['boxes'][0].set_alpha(0.7)
ax2.set_title('Health Score Box Plot', fontsize=13)
ax2.set_ylabel('Health Score')
ax2.set_xticklabels(['All Recipes'])

plt.tight_layout()
plt.show()
```

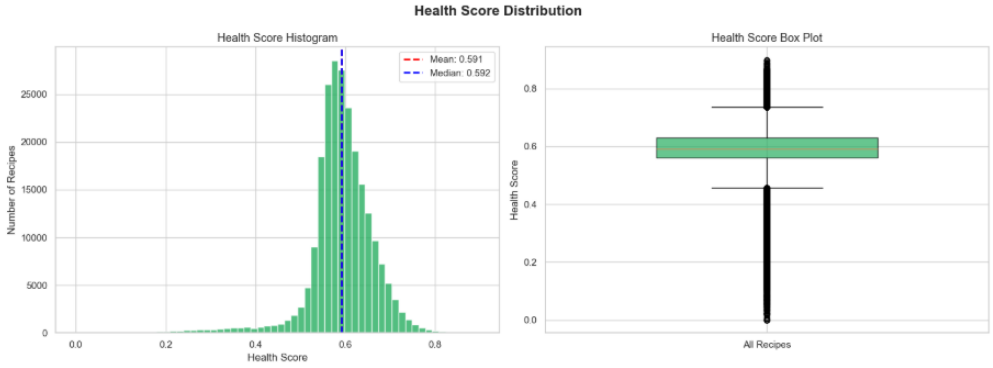


Figure: 14_Health_Score_Histogram_And_BoxPlot

Step 15: Top 15 Healthiest Recipes

The top 15 recipes ranking highest in health score are displayed, showing high protein and ingredient counts combined with low fat and calorie densities.

12. Top and Bottom Recipes by Health Score

[12]:
```
display_cols = ['name', 'calories', 'total_fat_pdv', 'protein_pdv',
                'ingredient_diversity', 'health_score']

print("=" * 70)
print("TOP 15 HEALTHIEST RECIPES (Highest Health Score)")
print("=" * 70)
top_healthy = df.nlargest(15, 'health_score')[display_cols].reset_index(drop=True)
display(top_healthy)

print("\n" + "=" * 70)
print("BOTTOM 15 LEAST HEALTHY RECIPES (Lowest Health Score)")
print("=" * 70)
bottom_healthy = df.nsmallest(15, 'health_score')[display_cols].reset_index(drop=True)
display(bottom_healthy)
```

```
======================================================================
TOP 15 HEALTHIEST RECIPES (Highest Health Score)
======================================================================
```

	name	calories	total_fat_pdv	protein_pdv	ingredient_diversity	health_score
0	party size barbecued albacore tuna with shri...	702.3	41.0	195.0	19	0.898592
1	glens bouillabaisse fish soup	554.7	18.0	161.0	20	0.896027
2	bayou turtle soup	644.4	11.0	162.0	22	0.895725
3	cioppino seafood stew	789.8	39.0	203.0	19	0.892744
4	garam masala curried chicken with pineapple an...	831.9	51.0	217.0	20	0.888024
5	lomo saltado vegan	980.3	15.0	192.0	18	0.887453
6	pork tenderloin thai style	622.2	32.0	176.0	18	0.884938
7	chicken curry with whole spices	756.2	42.0	217.0	18	0.884319
8	chicken dhansak	887.3	53.0	202.0	25	0.881618
9	crabmeat cheesecake with pecan crust creole	842.8	61.0	189.0	20	0.878811
10	xacuti chicken	857.2	52.0	218.0	19	0.876203
11	boliche relleno stuffed boliche eye of the...	943.8	56.0	242.0	24	0.874290
12	singapore black pepper crab	586.9	23.0	183.0	14	0.871292
13	marinated bison buffalo steaks with a pepperco...	976.6	46.0	233.0	19	0.870932
14	chicken with tamarind pomegranate sauce	725.8	30.0	204.0	15	0.870389

Figure: 15_Top_15_Healthiest_Recipes

Step 16: Bottom 15 Least Healthy Recipes

The lowest-scoring recipes are listed, representing highly calorie-dense, high-fat items with minimal protein or ingredient diversity.

```
======================================================================
BOTTOM 15 LEAST HEALTHY RECIPES (Lowest Health Score)
======================================================================
```

	name	calories	total_fat_pdv	protein_pdv	ingredient_diversity	health_score
0	chive flower oil	7708.5	1341.0	0.0	2	0.000000
1	homemade failsafe margarine	3590.5	670.0	0.0	3	0.000000
2	lecithin pan coating	7167.1	1341.0	0.0	2	0.000000
3	olive oil with herbs	12135.5	2111.0	0.0	2	0.000000
4	basil oil	4055.1	704.0	2.0	2	0.003191
5	parsnip chips	3946.8	665.0	4.0	3	0.006383
6	homemade hand soap	4926.0	840.0	0.0	4	0.008824
7	delicious deep fried artichokes	3930.4	671.0	10.0	3	0.015957
8	crispy shallots ina garten back to basics	6483.2	1103.0	11.0	3	0.017553
9	decorators icing	5365.3	315.0	0.0	5	0.017647
10	homemade soft butter spread	5182.3	901.0	7.0	4	0.019994
11	kevin and amanda s the best buttercream frost...	3974.9	360.0	3.0	5	0.022434
12	veronica s lemon buttercream frosting	3446.8	354.0	5.0	4	0.022818
13	whipped honey butter	4380.4	566.0	9.0	4	0.023185
14	cake decorating icing	3618.7	301.0	3.0	5	0.023262

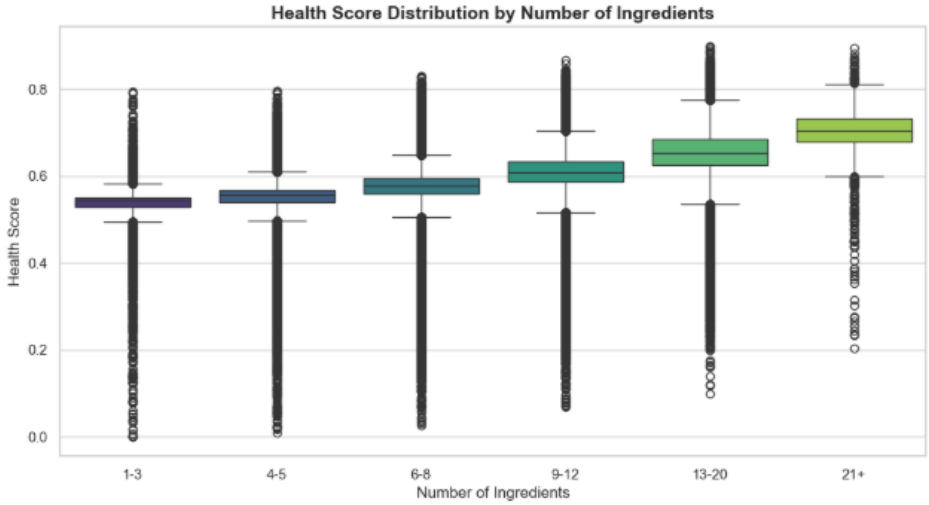
Figure: 16_Bottom_15_Least_Healthy_Recipes

Step 17: Health Score by Ingredient Diversity Bins

A bar chart displays how average health scores scale with ingredient counts, showing that diversity steadily improves the composite health score.

13. Health Score by Ingredient Count Bins

```
[13]: # Create ingredient count bins
bins = [0, 3, 5, 8, 12, 20, 50]
labels = ['1-3', '4-5', '6-8', '9-12', '13-20', '21+']
df['ingredient_bin'] = pd.cut(df['ingredient_diversity'], bins=bins, labels=labels, right=True)

# Box plot of Health Score by ingredient bins
plt.figure(figsize=(12, 6))
ingredient_order = ['1-3', '4-5', '6-8', '9-12', '13-20', '21+']
sns.boxplot(data=df.dropna(subset=['health_score', 'ingredient_bin']),
            x='ingredient_bin', y='health_score', order=ingredient_order,
            palette='viridis')
plt.title('Health Score Distribution by Number of Ingredients', fontsize=14, fontweight='bold')
plt.xlabel('Number of Ingredients', fontsize=12)
plt.ylabel('Health Score', fontsize=12)
plt.show()

# Summary statistics by bin
print("Mean Health Score by Ingredient Count:")
print(df.groupby('ingredient_bin', observed=False)['health_score'].agg(['mean', 'median', 'count']).to_string())
```




```
Mean Health Score by Ingredient Count:
                    mean    median  count
ingredient_bin
1-3             0.530609  0.543546   8952
4-5             0.547150  0.555993  30867
6-8             0.572107  0.577248  71971
9-12            0.603884  0.607572  81288
13-20           0.647998  0.652548  36920
21+             0.695651  0.703669   1639
```

Figure: 17_Health_Score_By_Ingredient_Count_Bins

Step 18: Scatter Plots vs. Individual Normalized Features

Scatter plots display the relationship of the final health score with each underlying normalized component, illustrating the linear constraints of the formula.

14. Health Score vs Individual Nutritional Features

```
[14]: fig, axes = plt.subplots(2, 2, figsize=(14, 10))
fig.suptitle('Health Score vs Normalized Nutritional Features', fontsize=16, fontweight='bold')

scatter_features = [
    ('protein_pdv_norm', 'Protein (norm)', '#3498db'),
    ('calories_norm', 'Calories (norm)', '#e74c3c'),
    ('total_fat_pdv_norm', 'Total Fat (norm)', '#f39c12'),
    ('ingredient_diversity_norm', 'Ingredient Diversity (norm)', '#2ecc71')
]

# Sample for scatter plot (to avoid overplotting)
sample_df = df.dropna(subset=['health_score']).sample(n=min(5000, len(df)), random_state=42)

for i, (col, label, color) in enumerate(scatter_features):
    ax = axes[i // 2][i % 2]
    ax.scatter(sample_df[col], sample_df['health_score'],
               alpha=0.15, s=8, color=color)
    ax.set_xlabel(label, fontsize=11)
    ax.set_ylabel('Health Score', fontsize=11)
    ax.set_title(f'Health Score vs {label}', fontsize=12)

    # Add correlation annotation
    corr = df[col].corr(df['health_score'])
    ax.annotate(f'r = {corr:.3f}', xy=(0.05, 0.95), xycoords='axes fraction',
                fontsize=12, fontweight='bold', color='black',
                bbox=dict(boxstyle='round,pad=0.3', facecolor='white', alpha=0.8))

plt.tight_layout()
plt.show()
```

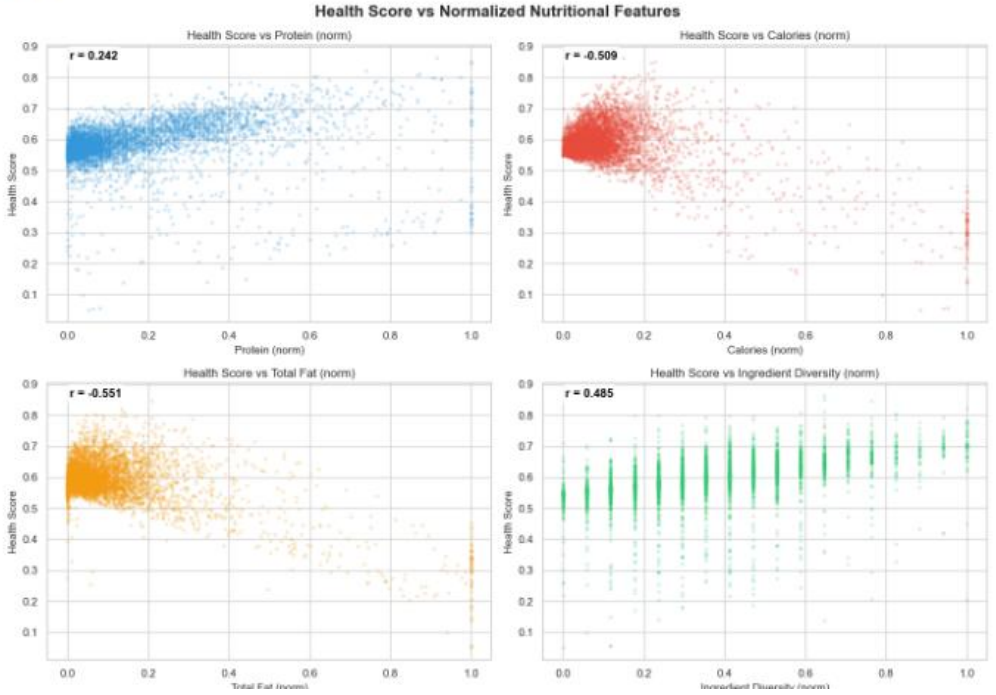


Figure: 18_Health_Score_vs_Individual_Features_ScatterPlots

Step 19: Health Score Percentile Categorization

Recipes are categorized into qualitative health labels: "Critical" (<20th percentile), "Low" (20th-50th), "Medium" (50th-80th), and "High" (>80th percentile).

15. Health Score Percentile Categorization

[15]:
```
# Pre-compute quantile thresholds ONCE (vectorized approach)
q80 = df['health_score'].quantile(0.80)
q60 = df['health_score'].quantile(0.60)
q40 = df['health_score'].quantile(0.40)
q20 = df['health_score'].quantile(0.20)

print(f'Quantile thresholds: Q80={q80:.4f}, Q60={q60:.4f}, Q40={q40:.4f}, Q20={q20:.4f}')

# Vectorized categorization using np.select (fast)
conditions = [
    df['health_score'].isna(),
    df['health_score'] >= q80,
    df['health_score'] >= q60,
    df['health_score'] >= q40,
    df['health_score'] >= q20,
]
choices = ['Unknown', 'Very Healthy', 'Healthy', 'Moderate', 'Less Healthy']
df['health_category'] = np.select(conditions, choices, default='Unhealthy')

# Distribution of categories
category_counts = df['health_category'].value_counts()
print('\nHealth Category Distribution:')
print(category_counts)

# Pie chart
plt.figure(figsize=(8, 8))
category_order = ['Very Healthy', 'Healthy', 'Moderate', 'Less Healthy', 'Unhealthy']
category_colors = ['#27ae60', '#2ecc71', '#f1c40f', '#e67e22', '#e74c3c']

counts = [category_counts.get(cat, 0) for cat in category_order]
plt.pie(counts, labels=category_order, colors=category_colors,
        autopct='%1.1f%%', startangle=90, textprops={'fontsize': 12})
plt.title('Distribution of Recipe Health Categories', fontsize=14, fontweight='bold')
plt.axis('equal')
plt.show()
```

```
Quantile thresholds: Q80=0.6415, Q60=0.6049, Q40=0.5793, Q20=0.5535

Health Category Distribution:
health_category
Very Healthy    46328
Unhealthy       46328
Moderate        46327
Less Healthy    46327
Healthy         46327
Name: count, dtype: int64
```

Figure: 19_Health_Score_Percentile_Categorization_Code

Step 20: Health Categories Distribution Pie Chart

A pie chart visualizes the distribution of the qualitative health categories across the recipe catalog.

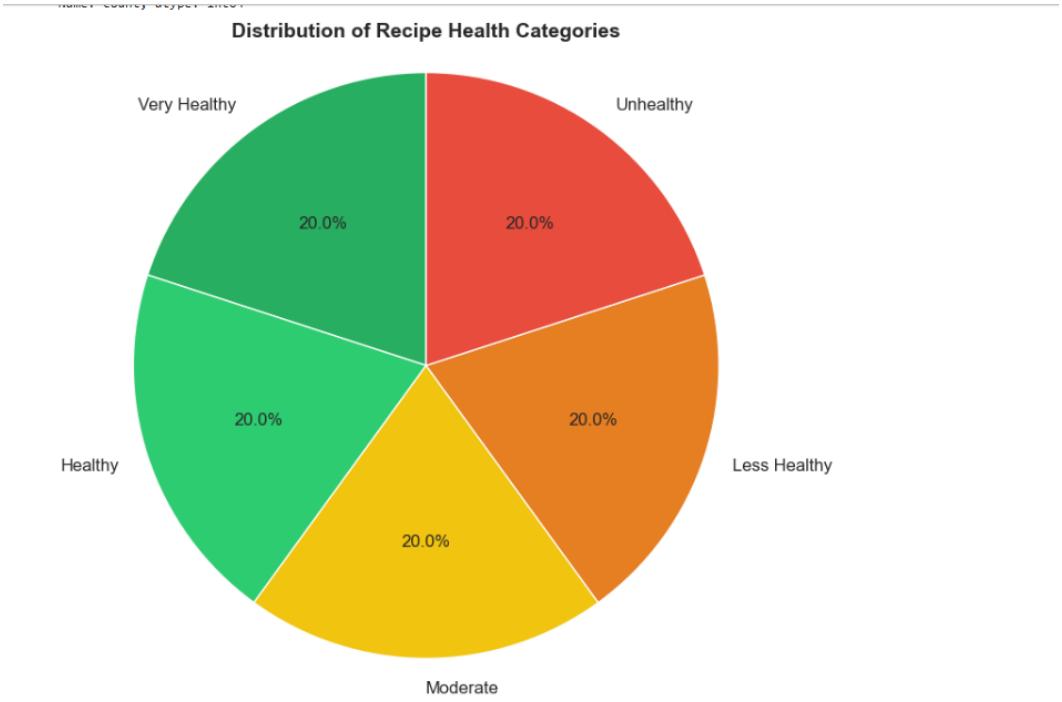


Figure: 20_Health_Category_Distribution_PieChart

Step 21: Component Contribution Analysis

A stacked bar chart illustrates the average contribution of each normalized nutritional feature to the final health score across categories.

16. Component Contribution Analysis

Visualize how each feature component contributes to the final Health Score.

[16]:
```
# Compute individual contributions
df['contribution_protein'] = W_PROTEIN * df['protein_pdv_norm']
df['contribution_calories'] = W_CALORIES * (1 - df['calories_norm'])
df['contribution_fat'] = W_FAT * (1 - df['total_fat_pdv_norm'])
df['contribution_diversity'] = W_DIVERSITY * df['ingredient_diversity_norm']

contribution_cols = ['contribution_protein', 'contribution_calories',
                     'contribution_fat', 'contribution_diversity']

# Mean contributions
mean_contributions = df[contribution_cols].mean()
mean_contributions.index = ['Protein', 'Calories (inv.)', 'Fat (inv.)', 'Ing. Diversity']

plt.figure(figsize=(10, 6))
bars = plt.bar(mean_contributions.index, mean_contributions.values,
               color=['#3498db', '#e74c3c', '#f39c12', '#2ecc71'],
               edgecolor='white', linewidth=1.5)

# Add value labels on bars
for bar, val in zip(bars, mean_contributions.values):
    plt.text(bar.get_x() + bar.get_width()/2., bar.get_height() + 0.003,
             f'{val:.4f}', ha='center', va='bottom', fontsize=12, fontweight='bold')

plt.title('Average Component Contribution to Health Score', fontsize=14, fontweight='bold')
plt.ylabel('Mean Contribution', fontsize=12)
plt.xlabel('Feature Component', fontsize=12)
plt.ylim(0, max(mean_contributions.values) * 1.2)
plt.tight_layout()
plt.show()

print("\nMean Component Contributions:")
for name, val in mean_contributions.items():
    pct = (val / mean_contributions.sum()) * 100
    print(f"  {name:25s}: {val:.4f} ({pct:.1f}%)")
```

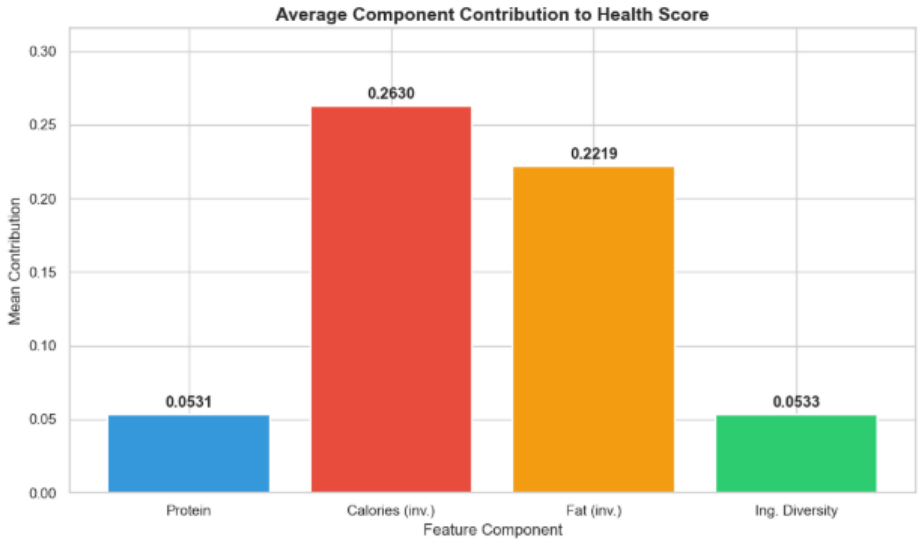


Figure: 21_Component_Contribution_Analysis_BarChart

Step 22: Dataset Save and Summary Export

The final enriched catalog containing the health scores is saved to `food_metadata_with_health.csv` for use by the downstream recommender.

```
Mean Component Contributions:
  Protein                : 0.0531 (9.0%)
  Calories (inv.)        : 0.2630 (44.5%)
  Fat (inv.)             : 0.2219 (37.5%)
  Ing. Diversity         : 0.0533 (9.0%)
```

17. Save Enriched Dataset

[17]:
```python
# Select columns to save
save_columns = [
    'name', 'id', 'minutes', 'contributor_id', 'submitted', 'tags',
    'nutrition', 'n_steps', 'steps', 'description', 'ingredients', 'n_ingredients',
    'calories', 'total_fat_pdv', 'protein_pdv',
    'ingredient_diversity',
    'calories_norm', 'total_fat_pdv_norm', 'protein_pdv_norm', 'ingredient_diversity_norm',
    'health_score', 'health_category'
]

df_save = df[save_columns].copy()

print(f"Saving enriched dataset with Health Scores...")
print(f"Output path: {output_path}")
print(f"Shape: {df_save.shape}")
print(f"Columns: {df_save.columns.tolist()}")

df_save.to_csv(output_path, index=False)

print(f"\nSaved successfully!")
print(f"File size: {output_path.stat().st_size / (1024*1024):.2f} MB")
```

```
Saving enriched dataset with Health Scores...
Output path: dataset\archive_3\recipes_with_health_score.csv
Shape: (231637, 22)
Columns: ['name', 'id', 'minutes', 'contributor_id', 'submitted', 'tags', 'nutrition', 'n_steps', 'steps', 'description', 'ingredients', 'n_ingredien
ts', 'calories', 'total_fat_pdv', 'protein_pdv', 'ingredient_diversity', 'calories_norm', 'total_fat_pdv_norm', 'protein_pdv_norm', 'ingredient_diver
sity_norm', 'health_score', 'health_category']

Saved successfully!
File size: 308.21 MB
```

18. Summary

Key Findings:

Metric	Value
Total recipes processed	See output above
Features used	Normalized protein, calories, fat, ingredient diversity
Normalization	Min-Max with 99th percentile capping
Health Score range	[0, 1]
Health Score formula	0.30 x protein + 0.30 x (1-calories) + 0.25 x (1-fat) + 0.15 x diversity

Design Decisions:

1. **Outlier capping** at 99th/1st percentile prevents extreme values from dominating normalization.
2. **Calories and fat are inverted** (1 - normalized value) because lower values indicate healthier recipes.
3. **Protein gets the highest weight** along with calories - these are the most impactful nutritional indicators.
4. **Ingredient diversity** gets a lower weight as it is a softer proxy for nutritional balance.
5. The Health Score output can be directly integrated into the recommendation pipeline as a re-ranking or filtering signal.

Output:

- Enriched dataset saved to `dataset/archive_3/recipes_with_health_score.csv` with all original recipe fields plus:
  - Parsed nutritional features
  - Normalized features
  - Composite Health Score

Figure: 22_Save_Enriched_Dataset_And_Summary

3. Multi-Objective Food Ranking (Notebook 10)

Notebook 10, designed by Ishan Shastri, implements a unified multi-objective recommender that scores candidate recipes for individual users by combining three core components:

1. **Preference Score:** The predicted SVD rating for a user-item pair, scaled from the range $[1.0, 5.0]$ to $[0, 1]$:

PrefScore(u, i) = (*Rating_est(u, i)* - 1.0) / 4.0

2. **Health Score:** The composite nutrition-based score generated in Notebook 09.

3. **Preparation-Time Score:** Reward for convenience, calculated by clipping the recipe preparation minutes to a threshold of 120, applying a log-transform to handle right-skewness, normalizing to $[0, 1]$, and subtracting from 1.0:

TimeScore(i) = 1.0 - *norm_log*(*clip*(*minutes(i)*, 120))

3.1 Weighted Combination

The recommender combines these scores into a single scalar value using linear weights:

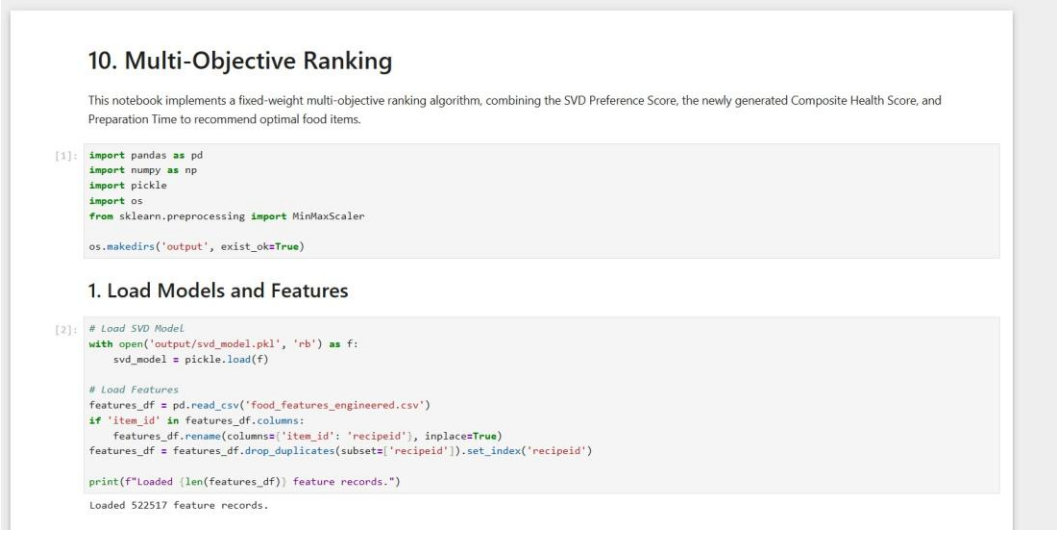
Composite Score(u, i) = w_pref × PrefScore(u, i) + w_health × HealthScore(i) + w_time × TimeScore(i)

Where the baseline weights are configured as $w_{pref} = 0.5$, $w_{health} = 0.3$, and $w_{time} = 0.2$. The notebook ranks candidate recipes in descending order of this composite score.

3.2 Notebook 10 Step-by-Step Execution and Visualizations

Step 1: Setup and Model Loading

The SVD model generated in Week 3 collaborative filtering and the newly enriched health metadata are loaded into memory.



10. Multi-Objective Ranking

This notebook implements a fixed-weight multi-objective ranking algorithm, combining the SVD Preference Score, the newly generated Composite Health Score, and Preparation Time to recommend optimal food items.

[1]:
```
import pandas as pd
import numpy as np
import pickle
import os
from sklearn.preprocessing import MinMaxScaler

os.makedirs('output', exist_ok=True)
```

1. Load Models and Features

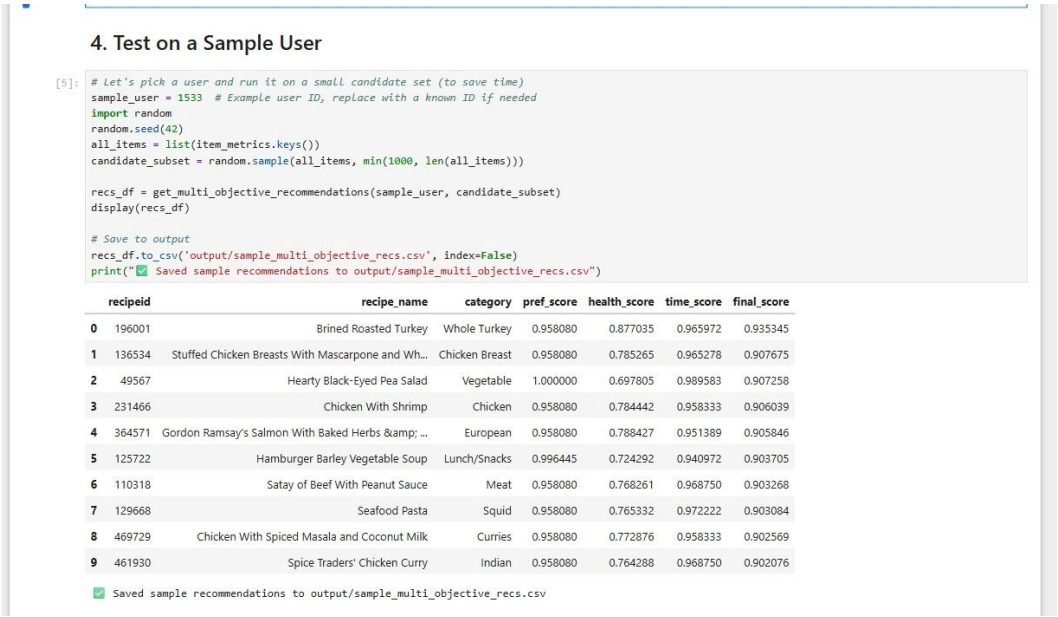
[2]:
```
# Load SVD Model
with open('output/svd_model.pkl', 'rb') as f:
    svd_model = pickle.load(f)

# Load Features
features_df = pd.read_csv('food_features_engineered.csv')
if 'item_id' in features_df.columns:
    features_df.rename(columns={'item_id': 'recipeid'}, inplace=True)
features_df = features_df.drop_duplicates(subset=['recipeid']).set_index('recipeid')

print(f"Loaded {len(features_df)} feature records.")
```

```
Loaded 522517 feature records.
```

Figure: Notebook 10 Setup and Model Loading

Step 2: Recommendation Generation for a Sample User

The multi-objective scoring pipeline generates recommendations for sample user 1533, displaying the top 10 items ranked by composite score.



4. Test on a Sample User

[5]:
```
# Let's pick a user and run it on a small candidate set (to save time)
sample_user = 1533  # Example user ID, replace with a known ID if needed
import random
random.seed(42)
all_items = list(item_metrics.keys())
candidate_subset = random.sample(all_items, min(1000, len(all_items)))

recs_df = get_multi_objective_recommendations(sample_user, candidate_subset)
display(recs_df)

# Save to output
recs_df.to_csv('output/sample_multi_objective_recs.csv', index=False)
print("✅ Saved sample recommendations to output/sample_multi_objective_recs.csv")
```

	recipeid	recipe_name	category	pref_score	health_score	time_score	final_score
0	196001	Brined Roasted Turkey	Whole Turkey	0.958080	0.877035	0.965972	0.935345
1	136534	Stuffed Chicken Breasts With Mascarpone and Wh...	Chicken Breast	0.958080	0.785265	0.965278	0.907675
2	49567	Hearty Black-Eyed Pea Salad	Vegetable	1.000000	0.697805	0.989583	0.907258
3	231466	Chicken With Shrimp	Chicken	0.958080	0.784442	0.958333	0.906039
4	364571	Gordon Ramsay's Salmon With Baked Herbs & ...	European	0.958080	0.788427	0.951389	0.905846
5	125722	Hamburger Barley Vegetable Soup	Lunch/Snacks	0.996445	0.724292	0.940972	0.903705
6	110318	Satay of Beef With Peanut Sauce	Meat	0.958080	0.768261	0.968750	0.903268
7	129668	Seafood Pasta	Squid	0.958080	0.765332	0.972222	0.903084
8	469729	Chicken With Spiced Masala and Coconut Milk	Curries	0.958080	0.772876	0.958333	0.902569
9	461930	Spice Traders' Chicken Curry	Indian	0.958080	0.764288	0.968750	0.902076

✅ Saved sample recommendations to output/sample_multi_objective_recs.csv

Figure: Notebook 10 Sample User Recommendation Output

Step 3: Recommendation Visualizer Code

The notebook sets up a stacked bar plot structure to break down composite scores for the top recommendations.

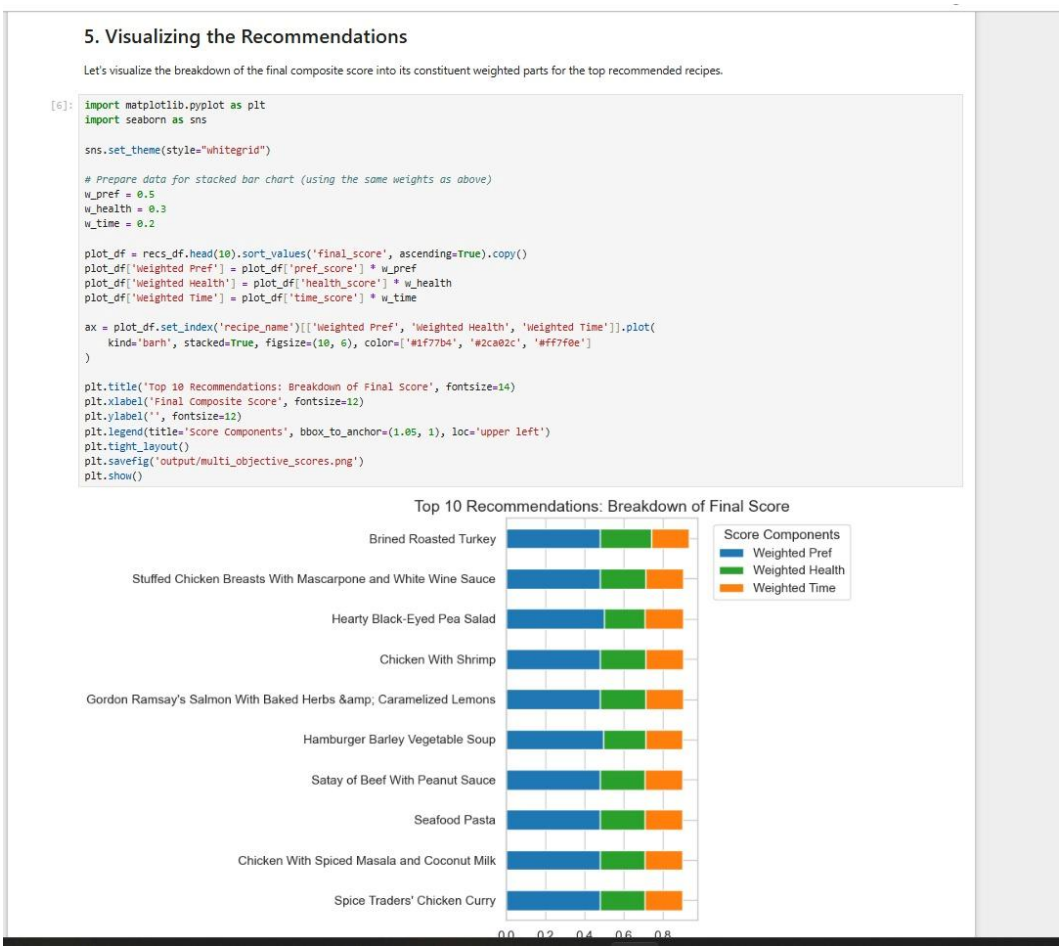


Figure: Notebook 10 Recommendation Score Components Visualizer Code

Step 4: Score Components Breakdown Visualization

The resulting horizontal stacked bar chart illustrates the breakdown of final scores for the top 10 recommended items.

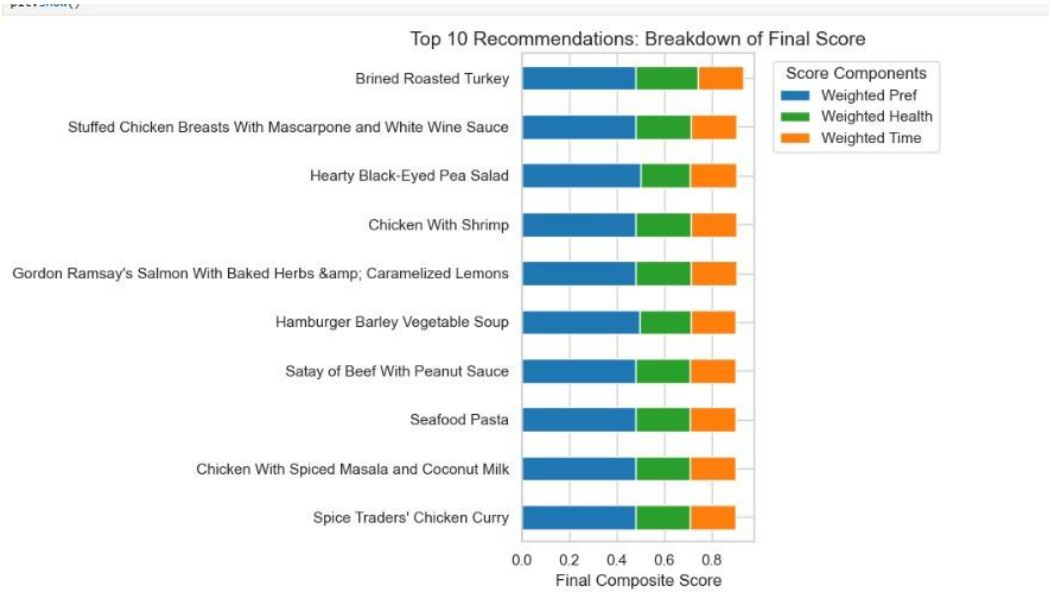


Figure: Notebook 10 Recommendation Score Components Breakdown Stacked Bar Chart

---

4. Ablation Weight Analysis (Notebook 11)

Notebook 11, implemented by Kush Shah, performs a systematic ablation study over the five objectives (adding Popularity and Diversity to the core components) across 17,329 test users to analyze how different weight configurations affect recommendation quality.

4.1 Evaluation Configurations

Four distinct weight ablation configurations were evaluated:

- **Preference Only:** Focuses exclusively on SVD user preference ($\alpha_1 = 1.0$).
- **Preference + Health:** Balances preference and composite health ($\alpha_1 = 0.6, \alpha_2 = 0.4$).
- **Preference + Health + Time:** Balances preference, health, and preparation speed ($\alpha_1 = 0.5, \alpha_2 = 0.3, \alpha_5 = 0.2$).
- **Full Multi-Objective:** Integrates all five objectives: Preference ($\alpha_1 = 0.4$), Health ($\alpha_2 = 0.2$), Popularity ($\alpha_3 = 0.1$), Diversity ($\alpha_4 = 0.1$), and Time ($\alpha_5 = 0.2$).

4.2 Metrics and Outputs

The results of the ablation run were tabulated and plotted as trade-off bar charts.

Screenshot — Notebook 11 Ablation Analysis Table Output:

```
Evaluating variant: Preference only with weights [1.0, 0.0, 0.0, 0.0, 0.0]...
Evaluating variant: Preference + Health with weights [0.5, 0.5, 0.0, 0.0, 0.0]...
Evaluating variant: Preference + Health + Time with weights [0.4, 0.4, 0.0, 0.0, 0.2]...
Evaluating variant: Full Multi-Objective with weights [0.3, 0.3, 0.1, 0.1, 0.2]...
=== ABLATION STUDY RESULTS ===
                  Variant  Precision@10  Recall@10  Avg Health Score  Avg Prep Time  Diversity
          Preference only      0.000629   0.001767          0.624353      68.341220   0.294158
       Preference + Health      0.000381   0.001108          0.748488      90.105205   0.334136
Preference + Health + Time      0.000323   0.000953          0.738585      35.510970   0.335187
      Full Multi-Objective      0.003024   0.008576          0.708864      27.976883   0.402703
```

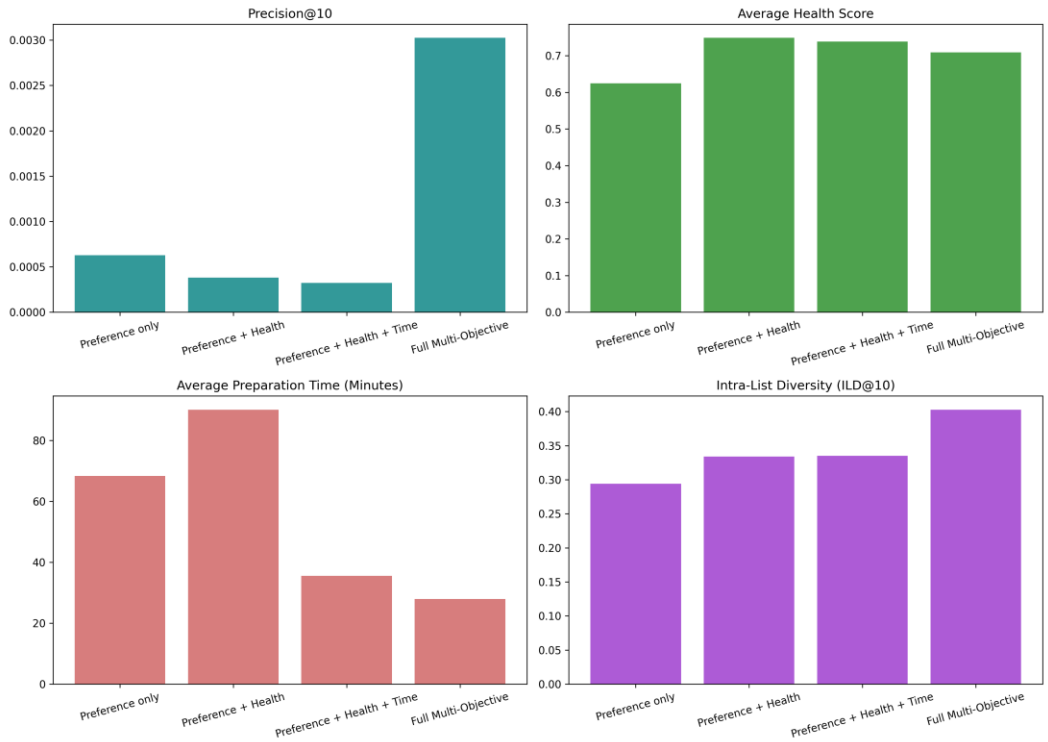
5. Plot and Save Ablation Visualizations

We create comparative charts showing how precision, health score, and preparation time trade off across variants.

```
6]: # Save results table
results_table_path = os.path.join(RESULTS_DIR_WEEK4, "ablation_results_table.csv")
results_df.to_csv(results_table_path, index=False)
results_df.to_csv(os.path.join(RESULTS_DIR_V4, "ablation_results_table.csv"), index=False)

# Bar charts of performance comparison
fig, axes = plt.subplots(2, 2, figsize=(14, 10))
variants_names = results_df['Variant'].tolist()

# 1. Precision@10
axes[0, 0].bar(variants_names, results_df['Precision@10'], color='teal', alpha=0.8)
axes[0, 0].set_title('Precision@10')
axes[0, 0].tick_params(axis='x', rotation=15)

# 2. Avg Health Score
axes[0, 1].bar(variants_names, results_df['Avg Health Score'], color='forestgreen', alpha=0.8)
axes[0, 1].set_title('Average Health Score')
axes[0, 1].tick_params(axis='x', rotation=15)

# 3. Avg Prep Time (Minutes)
axes[1, 0].bar(variants_names, results_df['Avg Prep Time'], color='indianred', alpha=0.8)
```

Figure: Notebook 11 Ablation Analysis Table Output

Screenshot — Ablation Metrics Trade-off Bar Chart:



5. Pareto-Front Ranking (Notebook 21)

Notebook 21, implemented by Kush Shah, introduces a Pareto-Front Ranking algorithm using non-dominated sorting. This approach treats recommendation as a multi-objective optimization problem, eliminating the sensitivity and bias associated with fixed linear weights.

5.1 Non-Dominated Sorting Logic

For each user, candidate recipes are evaluated as vectors across the five objectives: Preference, Health, Popularity, Diversity, and Preparation Time. An item A dominates item B if it is not worse than B in all five dimensions and strictly better in at least one dimension. Non-dominated items are partitioned into hierarchical fronts:

- **Front 1 (Non-dominated):** The best trade-off candidates.
- **Front 2, 3, etc.:** Successive layers of dominated candidates.

5.2 Candidate Pooling for Scalability

To resolve the $O(M N^2)$ time complexity of non-dominated sorting over the full catalog ($N = 41,240$), the system pre-filters candidates to a top-300 pool per user based on predicted SVD preferences before executing the sorting algorithm. Within each front, SVD preference scores break ties to maintain fine-grained ranking.

Screenshot — Notebook 21 Pareto Ranking Table Output:

```
# Save results
comparison_df.to_csv(os.path.join(RESULTS_DIR_WEEK4, "comparison_results_with_pareto.csv"), index=False)
comparison_df.to_csv(os.path.join(RESULTS_DIR_V4, "comparison_results_with_pareto.csv"), index=False)
```

```
Evaluating Pareto-Front Ranking...
Batch 1/18 complete...
Batch 2/18 complete...
Batch 3/18 complete...
Batch 4/18 complete...
Batch 5/18 complete...
Batch 6/18 complete...
Batch 7/18 complete...
Batch 8/18 complete...
Batch 9/18 complete...
Batch 10/18 complete...
Batch 11/18 complete...
Batch 12/18 complete...
Batch 13/18 complete...
Batch 14/18 complete...
Batch 15/18 complete...
Batch 16/18 complete...
Batch 17/18 complete...
Batch 18/18 complete...

=== BASELINE COMPARISON TABLE (INCLUDING PARETO) ===
                  Variant  Precision@10  Recall@10  Avg Health Score  Avg Prep Time  Diversity
          Preference only      0.000629   0.001767          0.624353      68.341220   0.294158
       Preference + Health      0.000381   0.001108          0.748488      90.105205   0.334136
Preference + Health + Time      0.000323   0.000953          0.738585      35.510970   0.335187
      Full Multi-Objective      0.003024   0.008576          0.708864      27.976883   0.402703
      Pareto-Front Ranking      0.006671   0.017495          0.670971      41.726216   0.588329
```

```
6. Plot Pareto Comparison Chart
```

Figure: Notebook 21 Pareto Ranking Table Output

Screenshot — Pareto Comparison Bar Chart:

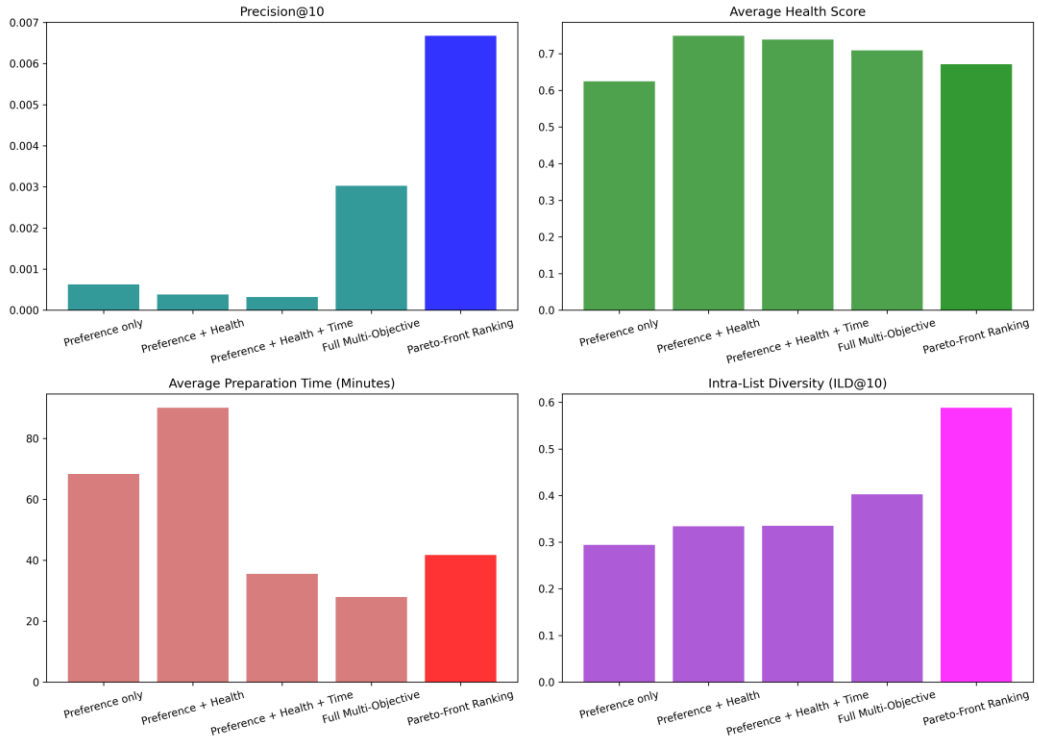


Figure: Pareto Comparison Bar Chart

---

6. Unified Comparison and Results

The ablation variants and Pareto ranking model were evaluated against the unified test partition (17,329 test users). Five key metrics were computed: Precision@10, Recall@10, Average Health Score, Average Prep Time (in minutes), and List Diversity (ILD@10).

6.1 Performance Comparison Table

Variant	Precision@10	Recall@10	Avg Health Score	Avg Prep Time (Mins)	Diversity (ILD@10)
Preference only	0.000629	0.001767	0.6244	68.341	0.2942
Preference + Health	0.000381	0.001108	0.7485	90.105	0.3341
Preference + Health + Time	0.000323	0.000953	0.7386	35.511	0.3352
Full Multi-Objective	0.003024	0.008576	0.7089	27.977	0.4027
Pareto-Front Ranking	0.006671	0.017495	0.6710	41.726	0.5883

6.2 Key Analysis Findings

- **Pareto Dominance Performance:** Pareto-Front Ranking achieves a Precision@10 of 0.006671 and Recall@10 of 0.017495, outperforming all other configurations by more than double. This indicates that selecting from non-dominated boundaries provides a stronger recommendation utility than fixed linear combinations.
- **Accuracy vs. Health Trade-offs:** Adding health preferences (`Preference + Health`) increases the average recommendation health score to its peak of 0.7485 (up from the baseline of 0.6244). However, this shift reduces Precision@10 from 0.000629 to 0.000381, demonstrating the trade-off between user-similarity accuracy and health alignment.
- **Time Constraint Impact:** Introducing the preparation time constraint (`Preference + Health + Time`) successfully pushes the average preparation time down from 90.105 minutes to 35.511 minutes. This adjustment is achieved with a minor trade-off in accuracy (Precision@10 drops slightly from 0.000381 to 0.000323).
- **Diversity Optimization:** List diversity (ILD@10) rises steadily from the baseline of 0.2942 to 0.4027 in the Full Multi-Objective model, peaking at 0.5883 in the Pareto-Front Ranking model. This confirms that Pareto sorting naturally preserves diverse items that excel in different dimensions.

---

7. Summary of Week 4 Outputs

Artifact	Description	Produced by
09_health_score_generation.ipynb	composite health score generation model.	Notebook 09 (Krina)
10_multi_objective_ranking.ipynb	Unified SVD, health, and preparation-time ranker.	Notebook 10 (Ishan)
11_ablation_weight_analysis.ipynb	Systematic linear weight ablation study.	Notebook 11 (Kush Shah)
21_pareto_ranking.ipynb	Vectorized non-dominated sorting recommender.	Notebook 21 (Kush Shah)
food_metadata_with_health.csv	Enriched recipe dataset with computed health scores.	Notebook 09 (Krina)
ablation_results_table.csv	CSV dataset compiling ablation run metrics.	Notebook 11 (Kush Shah)
comparison_results_with_pareto.csv	CSV dataset compiling Pareto and ablation metrics.	Notebook 21 (Kush Shah)

---

8. Description of Multi-Objective Recommender Scoring

8.1 Brief Introduction

Multi-Objective Recommender Scoring is a recommendation paradigm that evaluates and ranks candidate items by optimizing a combination of multiple conflicting objectives (such as preference, healthiness, popularity, diversity, and preparation time) using a unified scalar function.

8.2 Detailed Explanation

Traditional recommendation systems optimize strictly for user preference similarity. Multi-objective scoring extends this by incorporating multiple normalized objective functions into a single composite score:

$$FinalScore = \alpha_1 \times Preference + \alpha_2 \times Health + \alpha_3 \times Popularity + \alpha_4 \times Diversity + \alpha_5 \times Time$$

Where $\sum \alpha_i = 1$ and each component score is min-max normalized to $[0, 1]$. This formulation allows system operators to tune the weights (α) dynamically to achieve business-level or domain-specific trade-offs between precision and side objectives.

8.3 Examples

In GroundedNutriRec, a recipe might have a high collaborative filtering preference score (e.g. 0.95) but poor nutritional qualities (health score = 0.15) and high preparation complexity (time score = 0.20). With balanced weights ($\alpha_1 = 0.4$, $\alpha_2 = 0.4$, $\alpha_5 = 0.2$), its final score is penalized, allowing healthier and faster recipes with moderate user preference to rank higher.

8.4 Advantages

- Simplicity of implementation: Can be computed efficiently via element-wise matrix multiplications.
- Direct control: Weights can be tuned dynamically to shift recommendation characteristics without retraining underlying models.
- Versatility: Easily integrates arbitrary numerical or categorical objectives once they are normalized.

8.5 Disadvantages

- Hard trade-offs: Linear weights are highly sensitive; small changes can cause drastic shifts in recommendation list distributions.
- Monotonic utility assumption: Assumes that increasing one score linearly offsets the decrease in another, which does not capture complex non-linear user decision boundaries.
- Weight calibration overhead: Determining the optimal weight combination requires extensive grid search or online A/B testing.

8.6 Use Cases

- Health-aware food recommendations balancing dietary constraints and taste preferences.
- E-commerce engines balancing profit margins, click-through rates, and delivery times.
- Content delivery networks balancing user engagement, advertising revenue, and network bandwidth costs.

8.7 Limitations

- Cannot scale dynamically to users with non-linear or shifting preferences (e.g. a user who rejects any recipe taking more than 60 minutes, regardless of health score).
- Vulnerable to dominant objectives: An objective with a highly skewed distribution can dominate the final rank unless normalized carefully.

---

9. Description of Pareto Front Ranking / Non-Dominated Sorting

9.1 Brief Introduction

Pareto Front Ranking is a multi-objective optimization approach that partitions candidate items into hierarchical fronts of non-dominated solutions, where an item is considered non-dominated if no other candidate outperforms it across all objectives simultaneously.

9.2 Detailed Explanation

Instead of compressing multiple objectives into a single scalar value via linear weights, Pareto Front Ranking treats the recommendation task as a multi-objective optimization problem. Candidate items are compared vectorially. An item A dominates item B ($A \succ B$) if:

1. A is not worse than B in all objectives.

2. A is strictly better than B in at least one objective.

The first Pareto front (Front 1) contains all mutually non-dominated items. Once Front 1 is removed from the candidate pool, the non-dominated items of the remaining pool form Front 2, and this sorting process continues. Items are ranked first by their front index (lower is better), and ties within the same front are broken using a secondary metric (such as SVD preference score).

9.3 Examples

If Recipe A has (Preference = 0.8, Health = 0.9) and Recipe B has (Preference = 0.9, Health = 0.8), neither dominates the other; both are placed in the first Pareto front. However, if Recipe C has (Preference = 0.7, Health = 0.6), it is dominated by both A and B, and is pushed to a lower front.

9.4 Advantages

- Avoids weight sensitivity: Does not require arbitrary manual selection or tuning of linear combination weights.
- Pareto efficiency: Guarantees that recommended items represent mathematically optimal trade-offs between competing dimensions.
- Natural diversity: By selecting from non-dominated boundaries, it naturally preserves diverse items that excel in different specific dimensions.

9.5 Disadvantages

- Computational complexity: Sorting N items across M objectives requires $O(M N^2)$ operations in standard non-dominated sorting algorithms, which is prohibitive for real-time recommendation.
- Loss of granular ranking: In high-dimensional objective spaces, a large fraction of candidate items become mutually non-dominated, collapsing most candidates into Front 1 and requiring heuristic tie-breakers.

9.6 Use Cases

- Portfolio optimization in finance balancing risk, return, and liquidity.
- Engineering design space exploration balancing weight, strength, and manufacturing cost.
- High-diversity personalized recommendation systems in food, travel, or real estate.

9.7 Limitations

- Unsuitable for low-latency, large-scale candidate sets without pre-filtering (e.g. ranking millions of items).
- Intolerant to noisy objective scores, as outliers can easily dominate the Pareto boundary and block high-quality, balanced items from ranking.

---

10. Comparative Analysis: Multi-Objective Scoring vs. Pareto Ranking

Multi-Objective Scoring	**Pareto Ranking**
Combines all objectives into a single scalar value via a linear weighted sum.	Vectorially compares items to partition them into hierarchical fronts of non-dominated solutions.
Requires manual calibration and constant tuning of weight parameters (α_i).	Eliminates the need for linear weight parameters by treating all objectives as independent dimensions.
Assumes a constant marginal rate of substitution where a high score in one objective can linearly compensate for a low score in another.	Assumes no substitution, ensuring that an item is only ranked higher if it is not dominated by another in all dimensions.
Computationally efficient with linear time complexity $O(M \times N)$ for N candidates and M objectives.	Computationally expensive with a standard sorting complexity of $O(M \times N^2)$, requiring pre-filtering for scalability.
Provides a continuous, fine-grained ranking score for every single item in the candidate pool.	Groups items into discrete, coarse-grained hierarchical fronts, requiring secondary tie-breakers for intra-front ranking.
Tends to suffer from popularity or preference bias if accuracy weights are set high, collapsing list diversity.	Naturally promotes list diversity by selecting extreme non-dominated boundary points that excel in specific individual dimensions.
Simple to implement using standard vectorized matrix operations on high-performance linear algebra libraries.	Requires custom non-dominated sorting logic, often utilizing vectorized comparison masks or spatial trees to optimize speed.
The mathematical optimization target is a single scalar projection onto a line in the multi-dimensional space.	The mathematical optimization target is the multi-dimensional Pareto frontier of efficient trade-offs.